Function Prototype in C

Last Updated : 29 Sep, 2023

The C function prototype is a statement that tells the compiler about the function's name, its return type, numbers and data types of its parameters. By using this information, the compiler cross-checks function parameters and their data type with function definition and function call.

Function prototype works like a function declaration where it is necessary where the function reference or call is present before the function definition but optional if the function definition is present before the function call in the program.

Syntax

```
return_type function_name(parameter_list);
```

where,

- **return_type**: It is the data type of the value that the function returns. It can be any data type int, float, void, etc. If the function does not return anything, void is used as the return type.
- **function_name**: It is the identifier of the function. Use appropriate names for the functions that specify the purpose of the function.
- **parameter_list**: It is the list of parameters that a function expects in parentheses. A parameter consists of its data type and name. If we don't want to pass any parameter, we can leave the parentheses empty.

For example,

```
int func(int, char, char *, float);
```

Example:

Program to illustrate the Function Prototype

C

```c
// C program to illustrate the function prototye
#include <stdio.h>

// Function prototype
float calculateRectangleArea(float length, float width);

int main()
{
    float length = 5.0;
    float width = 3.0;

    // Function call
    float area = calculateRectangleArea(length, width);

    printf("The area of the rectangle is: %.2f\n", area);

    return 0;
}

// Function definition
float calculateRectangleArea(float length, float width)
{
    return length * width;
}
```

Output

```
The area of the rectangle is: 15.00
```

What if the function prototype is not specified?

If we ignore the function prototype, a program may compile with a warning, it may give errors and may work properly. But sometimes, it will give strange output and it is very hard to find such programming mistakes.

Example: Program without a function prototype

The below code opens a file specified by the command-line argument, checks if the file exists, and then closes the file. The prototype of strerror function is not included in the code.

C

```c
// C code to check if a file exists or not.
// Prototype of strerror function is not included in the
// code.

#include <errno.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
    FILE* fp;

    fp = fopen(argv[1], "r");
    if (fp == NULL) {
        fprintf(stderr, "%s\n", strerror(errno));
        return errno;
    }

    printf("file exist\n");

    fclose(fp);

    return 0;
}
```

Let us provide a filename, which does not exist in a file system, and check the output of the program on x86_64 architecture.

Output

```
[narendra@/media/partition/GFG]$ ./file_existence hello.c
Segmentation fault (core dumped)
```

Explanation

The above program checks the existence of a file, provided from the command line, if a given file exists, then the program prints “file exists”, otherwise it

prints an appropriate error message.

Why did this program crash, instead it should show an appropriate error message. This program will work fine on x86 architecture but will crash on x86_64 architecture. Let us see what was wrong with the code. Carefully go through the program, deliberately I haven't included the prototype of the "strerror()" function. This function returns a "pointer to the character", which will print an error message which depends on the errno passed to this function.

Note that x86 architecture is an ILP-32 model, which means integers, pointers, and long are 32-bit wide, that's why the program will work correctly on this architecture. But x86_64 is the LP-64 model, which means long and pointers are 64-bit wide. *In C language, when we don't provide a prototype of a function, the compiler assumes that the function returns an integer*. In our example, we haven't included the "string.h" header file (strerror's prototype is declared in this file), that's why the compiler assumed that the function returns an integer. But its return type is a pointer to a character.

In x86_64, pointers are 64-bit wide and integers are 32-bit wide, that's why while returning from a function, the returned address gets truncated (i.e. 32-bit wide address, which is the size of integer on x86_64) which is invalid and when we try to dereference this address, the result is a segmentation fault.

Now include the "string.h" header file and check the output, the program will work correctly and gives the following output.

```
[narendra@/media/partition/GFG]$ ./file_existence hello.c
No such file or directory
```

Consider one more example

This code demonstrates the process of dynamic memory allocation using the malloc function, but the prototype of the malloc function is not included.

C

```
// This code dynamically allocates memory for an integer
// using the malloc function but prototype of malloc
// function is not included in the code
```

```c
#include <stdio.h>

int main(void)
{
    int* p = malloc(sizeof(int));

    if (p == NULL) {
        perror("malloc()");
        return -1;
    }

    *p = 10;
    free(p);

    return 0;
}
```

Explanation

The above code will work fine on the IA-32 model but will fail on the IA-64 model. The reason for the failure of this code is we haven't included a prototype of the malloc() function and the returned value is truncated in the IA-64 model.

FAQs on Function Prototype

Q1. Difference between Function Declaration and Function Prototype

Answer:

Following are some differences between Function and Function Prototype:

Function Declaration	*Function Prototype*
Function Declaration is used to tell the existence of a function.	*The function prototype tells the compiler about the existence and signature of the function.*

Function Declaration	***Function Prototype***
A function declaration is valid even with only function name and return type.	*A function prototype is a function delcaration that provides the function's name, return type, and parameter list without including the function body.*
Typically used in header files to declare functions.	*Used to declare functions before their actual definitions.*
Syntax: *return_type function_name();*	***Syntax:*** *return_type function_name(parameter_list);*

This article is compiled by **Narendra Kangralkar**.